Universidad Nacional de Tucumán
Facultad de Ciencias Exactas y Tecnología



Proyecto: Simulación de Probabilidad Colectiva, Paradoja de Monty Hall

Métodos Numéricos II

Licenciatura en Informática

Integrantes:

Flores, Hector Adrian
Girbau Alfaro, Maria Joaquina
Gramajo, Juan Ignacio

2026

1. Resumen

Este informe presenta el análisis probabilístico del problema de Monty Hall generalizado a K puertas y la implementación serial y paralela (C + MPI) de una simulación de Monte Carlo para evaluar las estrategias de mantener o cambiar la elección inicial. Se expone la formulación matemática del método, el análisis de convergencia del error estocástico y la evaluación del rendimiento paralelo en un servidor de alto desempeño con dos procesadores AMD EPYC 7B12 de 64 núcleos cada uno (128 núcleos físicos en total), del cual se dispuso de hasta 96 núcleos.

Los resultados confirman que la precisión de la simulación mejora de manera consistente al aumentar el número de muestras N, con un error que decrece a una tasa de $O(1/\sqrt{N})$, y validan las probabilidades teóricas exactas P(Mantener) = $1/K$ y P(Cambiar) = $(K-1)/K$ en pruebas con K = 3, 10 y 100 puertas, donde el presentador abre $K-2$ puertas con cabra. En cuanto al desempeño, para cargas masivas ($N = 10^{10}$ simulaciones) la versión paralela con 96 núcleos reduce el tiempo de ejecución de 215.93 s (≈3.6 min) en la versión serial a 2.30 s, lo que equivale a un speedup de 93.95× y una eficiencia paralela del 97.86 %. Asimismo, para problemas de menor tamaño se observa una región de saturación al pasar de 64 a 96 núcleos, consistente con el cruce entre los dos sockets (dominios NUMA) y la interconexión Infinity Fabric.

2. Introducción

2.1 ¿Qué es el Método de Monte Carlo?

El método Monte Carlo fue inventado por John von Neumann y Stanislaw Ulam durante la Segunda Guerra Mundial para mejorar la toma de decisiones en condiciones inciertas.

Utilizan muestreo aleatorio repetido para obtener resultados numéricos en problemas que, en esencia, son deterministas. Los programas informáticos utilizan este método para analizar datos pasados y predecir una serie de resultados futuros en función de una elección de acción. Cuando se completa una simulación de Monte Carlo, se obtiene un rango de resultados posibles con la probabilidad de que ocurra cada resultado.

2.2 ¿Cómo funciona el Método de Monte Carlo?

Construye un modelo de posibles resultados aprovechando una distribución de probabilidad, para cualquier variable que tenga incertidumbre inherente. Luego, vuelve a calcular los resultados una y otra vez, cada vez mediante la repetición de experimentos aleatorios. En un experimento típico de Monte Carlo, este ejercicio se puede repetir miles de veces para producir una gran cantidad de resultados probables. La frecuencia relativa de un evento se utiliza como estimación de su probabilidad y, a medida que aumenta el número de ensayos independientes, tiende a estabilizarse alrededor del valor teórico. La incertidumbre de muestreo se resume mediante el error estándar, que decrece proporcionalmente a $1/\sqrt{N}$ para una probabilidad fija.

2.3 ¿Cuál es la paradoja de Monty Hall?

El problema de Monty Hall es un ejemplo clásico de probabilidad condicional. En su versión habitual hay tres puertas: detrás de una se encuentra un premio y detrás de las otras dos, premios no deseados. El participante elige una puerta; luego, un presentador que conoce la ubicación del premio abre una puerta distinta que necesariamente no contiene el premio y ofrece la posibilidad de cambiar la elección. Bajo estas reglas, mantener gana si la elección inicial era correcta, mientras que cambiar gana si la elección inicial era incorrecta. El problema fue formulado en la literatura estadística por Selvin (1975) y se popularizó a partir de la columna de Marilyn vos Savant (1990).

La intuición puede sugerir que, después de abrirse una puerta, las dos restantes tienen la misma probabilidad. Sin embargo, la información aportada por el presentador no es una eliminación aleatoria: su acción está condicionada por el conocimiento del premio. Con tres puertas, la probabilidad inicial de acertar es 1/3 y la de no acertar es 2/3. La estrategia de mantener conserva la probabilidad inicial, mientras que la estrategia de cambiar aprovecha el caso en que la primera elección era incorrecta. Para una generalización con K puertas, si el presentador abre $K-2$ puertas sin premio y deja una única alternativa a la elección inicial, las probabilidades son $P(\text{mantener}) = 1/K$ y $P(\text{cambiar}) = (K-1)/K$.

El objetivo general de este trabajo es contrastar las probabilidades teóricas de Monty Hall con estimaciones empíricas y evaluar el rendimiento de una implementación paralela. Los objetivos específicos son: (a) comparar las estrategias de mantener y cambiar para distintos valores de K; (b) estudiar la variación del error al aumentar N; (c) analizar el tiempo de ejecución, el speedup y la eficiencia al variar la cantidad de procesos MPI; y (d) identificar las limitaciones del modelo y de los datos disponibles.

3. Datos y métodos

3.1 Datos experimentales

División del trabajo: El número total de simulaciones se divide equitativamente entre los procesos disponibles, implementando un balanceador de carga que maneja el residuo de la división para asegurar que no se pierda ninguna iteración.
Cada registro incluye: número de puertas, cantidad total de simulaciones, procesos, tiempo mediano en segundos, speedup, eficiencia porcentual, frecuencia de victoria al mantener y al cambiar, error absoluto porcentual, error relativo porcentual, error estocástico (SE) y cantidad de repeticiones.

3.2 Modelo y procedimiento computacional

Nuestro código inicializa MPI y obtiene el identificador de cada proceso (rank) y la cantidad total de procesos. Divide N entre los procesos y asigna el residuo al último proceso. Cada proceso inicializa una semilla pseudoaleatoria dependiente de time(NULL) y del rank. En cada iteración genera una puerta para el premio y una puerta para la elección inicial, ambas uniformemente mediante rand_r(&seed) % K. Si ambas coinciden, incrementa el contador local de victorias al mantener. Luego MPI_Reduce suma los contadores locales en el proceso raíz. El número de victorias al cambiar se calcula como N menos las victorias al mantener.

El programa mide el tiempo transcurrido mediante MPI_Wtime, con barreras de sincronización alrededor de la región de medición. Calcula las frecuencias empíricas, las probabilidades teóricas, el error absoluto y el error relativo porcentual. También calcula el error estándar teórico de la estimación de mantener como $SE = 100\sqrt{[p(1-p)/N]}$, donde $p = 1/K$. Todos estos datos son guardados en un archivo CSV el cual además proporciona speedup y eficiencia; se analizan como métricas reportadas por el proceso de benchmark.

3.3 Supuestos y métricas

La equivalencia entre el contador complementario y la estrategia de cambiar depende de la regla del presentador: debe conocer la ubicación del premio, no abrir la puerta elegida y revelar todas las puertas sin premio necesarias para dejar una única alternativa cerrada. El código no simula explícitamente estas aperturas ni una decisión del presentador; representa directamente el resultado de cambiar como el complemento de acertar en la primera elección. Para $K > 3$, supone implícitamente que el presentador deja una sola alternativa.

El error absoluto que se presenta en el CSV se expresa en puntos porcentuales respecto de la probabilidad teórica de mantener. El error estándar es una medida de dispersión esperada por muestreo, no un error observado ni una garantía de que cada ejecución caiga dentro de un intervalo específico.

4. Resultados (Link Repositorio GITHUB)

4.1 Contraste entre probabilidad teórica y empírica

Para N = 10^{10} y un proceso, la frecuencia empírica de mantener se aproxima a 1/K. La frecuencia empírica de cambiar es su complemento y, por tanto, se aproxima a (K−1)/K. La Tabla 1 resume los valores del CSV y los compara con la teoría.

K	Mantener teórico	Mantener empírico	Cambiar teórico	Cambiar empírico	Error abs. (pp)
3	33.333333%	33.331297%	66.666667%	66.668703%	0.002036
10	10.000000%	10.001820%	90.000000%	89.998180%	0.001820
100	1.000000%	0.999987%	99.000000%	99.000013%	0.000013

En los tres casos la diferencia es pequeña en términos absolutos. La probabilidad de cambiar resulta cercana al complemento teórico correspondiente.

4.2 Efecto del número de simulaciones

Los datos para un proceso muestran que las estimaciones se estabilizan al aumentar N. Con N = 100, las fluctuaciones son visibles, en particular para K = 100, donde la frecuencia empírica de mantener es 0,4 % frente al 1 % teórico. Con N = 10^6, la estimación para K = 100 ya es 0,999630 %, y con N = 10^{10} es 0,999987 %. Esta tendencia es compatible con la ley de los grandes números. No obstante, la convergencia no tiene por qué ser monótona: una muestra mayor no exige que cada estimación individual esté más cerca que la anterior.

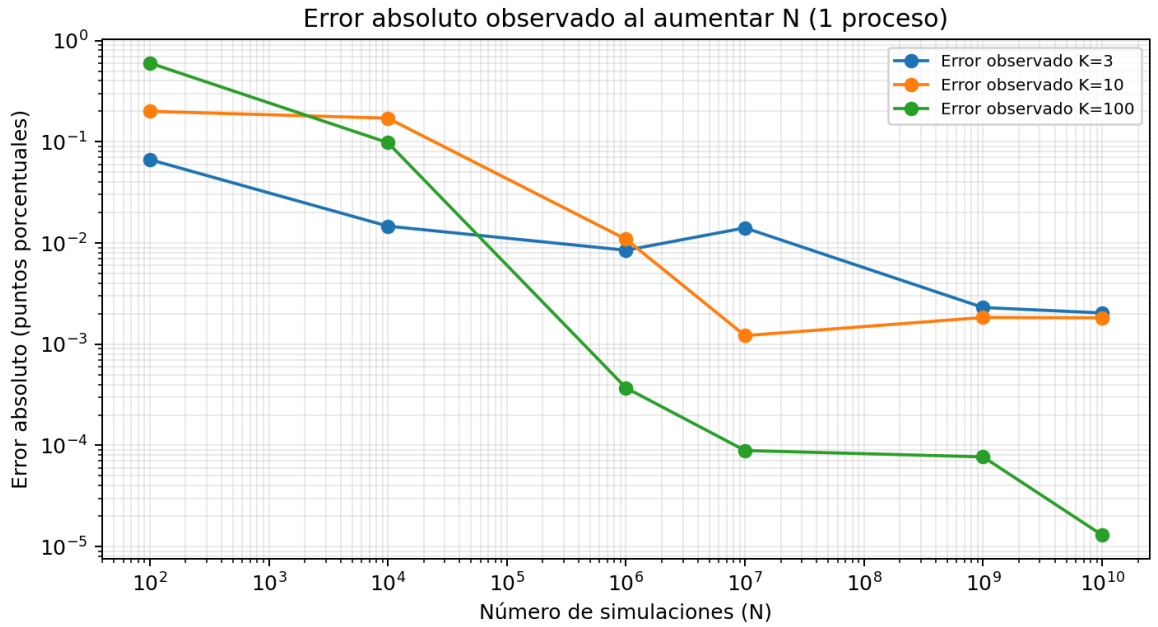


Figura 2. Error absoluto registrado al aumentar N, con un proceso. Escala logarítmica en ambos ejes.

El error estándar teórico registrado para $N = 10^{10}$ es aproximadamente 0,000471 puntos porcentuales para K = 3, 0,000300 para K = 10 y 0,000100 para K = 100. La magnitud depende de $p(1-p)$: para una proporción de éxito muy pequeña, como 1 % en K = 100, el error estándar absoluto de la frecuencia porcentual es menor que en K = 3. **Esto no significa que toda medida relativa del error sea necesariamente menor.**

4.3 Rendimiento Paralelo

En el benchmark de $N = 10^{10}$, el tiempo mediano disminuye al aumentar los procesos. Para K = 3 pasa de 215,729 s con un proceso a 2,299 s con 96 procesos; para K = 10, de 176,260 s a 1,866 s; y para K = 100, de 156,191 s a 1,645 s. El CSV reporta speedups de 93,8502, 94,4721 y 94,9432, respectivamente, para 96 procesos.

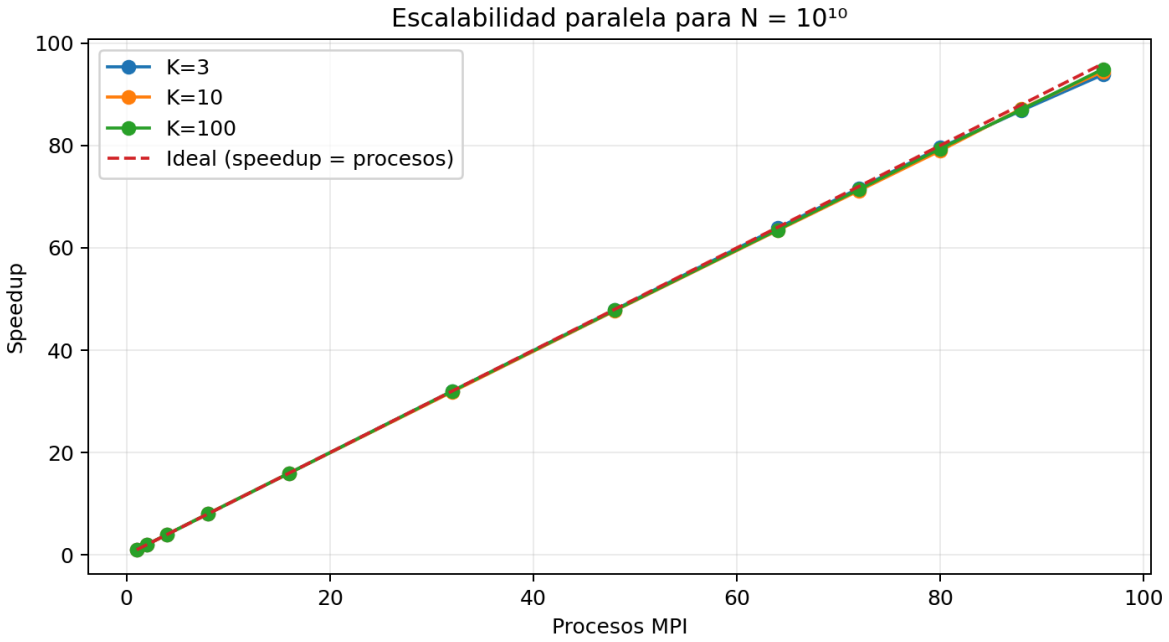


Figura 3. Speedup reportado según procesos MPI para $N = 10^{10}$.

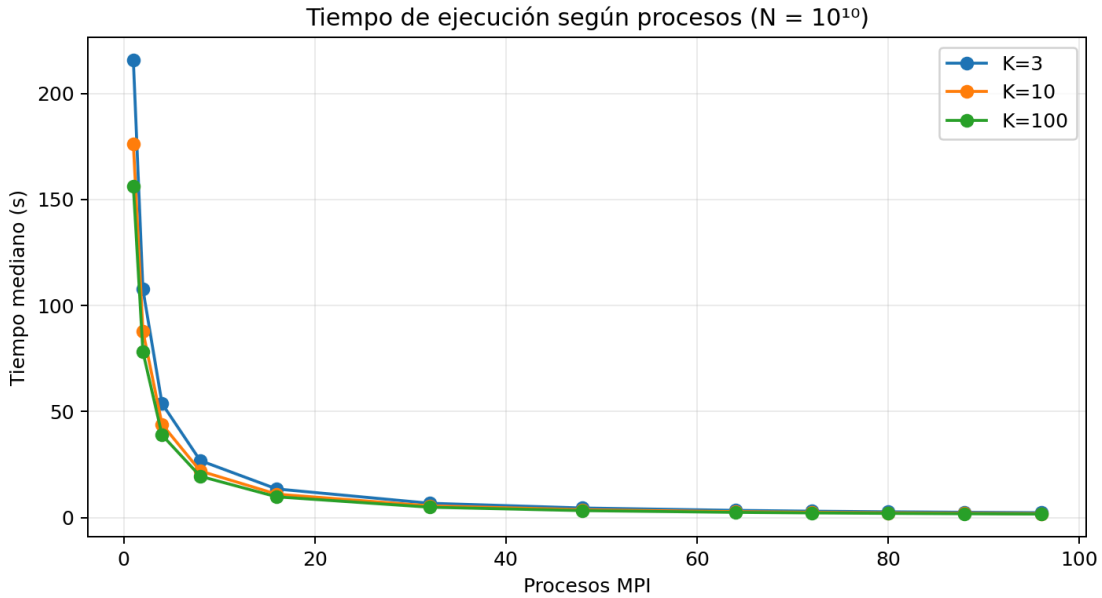


Figura 4. Tiempo mediano de ejecución según procesos MPI para $N = 10^{10}$.

K	Tiempo 1 proceso (s)	Tiempo 96 procesos (s)	Speedup (96)	Eficiencia (96)
3	215.729	2.299	93.85	97.76%
10	176.260	1.866	94.47	98.41%
100	156.191	1.645	94.94	98.90%

Los speedups se acercan al valor ideal de 96, aunque no lo alcanzan; las eficiencias registradas para 96 procesos son 97,76 % para K = 3, 98,41 % para K = 10 y 98,90 % para K = 100. Son valores altos en las condiciones del benchmark. Sin información sobre el hardware y el método exacto de medición, estos resultados deben describirse como rendimiento reportado por el CSV y no como una comparación reproducible entre máquinas. Algunas eficiencias superan ligeramente el 100 % en configuraciones pequeñas o intermedias; esto puede ocurrir por variabilidad de medición, granularidad del reloj o efectos del benchmark, y no implica por sí solo una aceleración física imposible.

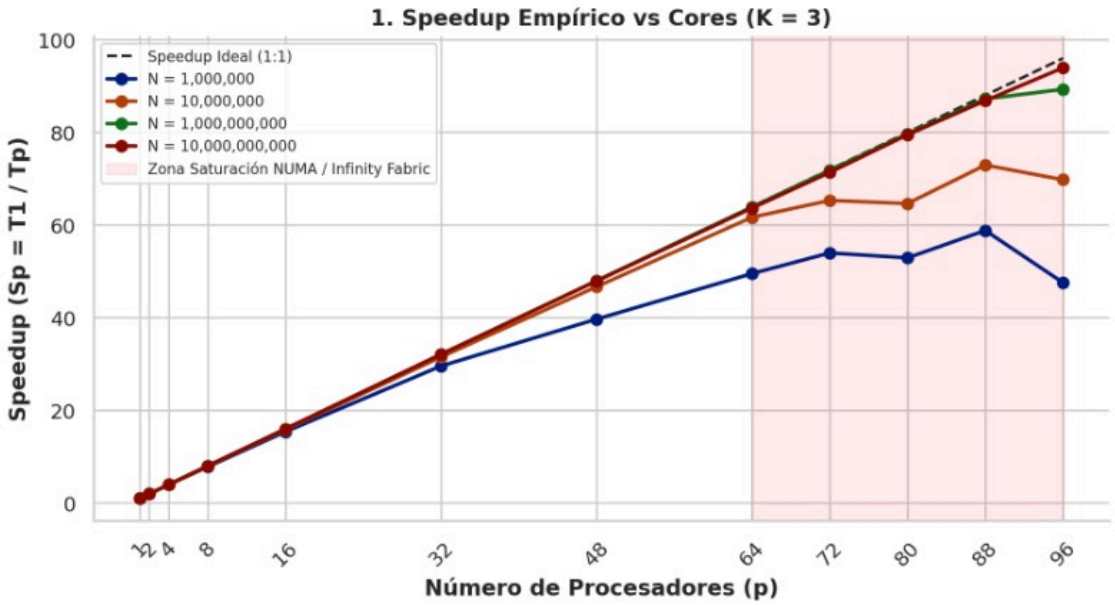


Figura 5. Speed up Empírico con 3 puertas y distinta cantidad de simulaciones

5. Discusión y limitaciones

Los resultados apoyan la conclusión probabilística principal: cambiar es más favorable que mantener bajo las reglas estándar del problema, y la ventaja aumenta con K. Para K = 3, cambiar tiene probabilidad teórica de 2/3; para K = 10, 9/10; y para K = 100, 99/100. Las frecuencias empíricas de las simulaciones grandes son coherentes con esos valores. La experimentación también muestra que el trabajo es paralelizable: la mayor parte de cada iteración es independiente y los contadores locales se agregan mediante una reducción colectiva.

Hay, sin embargo, limitaciones que deben tenerse en cuenta. **Primero**, el código no simula la apertura de puertas por parte del presentador: computa la estrategia de cambiar como el complemento de la victoria al mantener. Esto resulta válido bajo el supuesto de una única alternativa final, pero no permite estudiar políticas alternativas del presentador ni versiones distintas del juego. **Segundo**, la semilla se deriva del tiempo en segundos y del rank; la combinación reduce la probabilidad de semillas idénticas entre procesos, pero no constituye una garantía de independencia estadística ni un generador aleatorio de alta calidad para todos los fines. **Tercero**, el reparto asigna el

residuo al último proceso, lo que puede producir una carga ligeramente desigual cuando N no es divisible por el número de procesos.

Cuarto, las ejecuciones de tamaño pequeño duran microsegundos o fracciones muy pequeñas de segundo, por lo que las medidas de tiempo pueden ser sensibles a la resolución del reloj y al ruido del sistema. Finalmente, el error estándar del código se calcula para la probabilidad de mantener; no representa directamente el error estándar de todas las métricas reportadas ni un intervalo de confianza completo.

6. Conclusiones

Se cumplió el objetivo de contrastar las probabilidades teóricas de Monty Hall con resultados de Monte Carlo para distintas cantidades de puertas y simulaciones. En las configuraciones con N = 10^{10}, las frecuencias observadas se aproximan estrechamente a 1/K para mantener y a (K−1)/K para cambiar. La simulación, por tanto, respalda la conclusión de que cambiar es la estrategia más conveniente bajo las reglas estándar del problema.

El análisis del tamaño muestral es consistente con la teoría del método de Monte Carlo: al crecer N, la variabilidad de la estimación disminuye aproximadamente en proporción a $1/\sqrt{N}$, aunque el error observado puede subir o bajar entre tamaños de muestra consecutivos. La generalización a K = 3, 10 y 100 permite observar que la probabilidad de ganar cambiando aumenta desde aproximadamente 66,67 % hasta 90 % y 99 %.

En cuanto al rendimiento, los benchmarks muestran una reducción importante del tiempo al aumentar los procesos MPI. Para N = 10^{10} y 96 procesos, el speedup reportado está entre 93,85 y 94,94, dependiendo de K. Este resultado indica buena escalabilidad en los datos suministrados, aunque su reproducibilidad requiere documentar el entorno de ejecución y el protocolo de medición.

Además, como posible mejora futura, se propone evaluar un generador pseudoaleatorio diseñado para entornos paralelos, y verificar la reproducibilidad y la calidad estadística de las secuencias utilizadas por los procesos MPI. Esta evaluación permitiría estudiar posibles dependencias entre flujos y mejorar la trazabilidad de los experimentos.

7. Bibliografía

- IBM. Monte Carlo. Leer Artículo

- Amazon. Monte Carlo. Leer Artículo

- Grinstead, C. M., & Snell, J. L. (1997). Introduction to Probability. American Mathematical Society. Consultar libro gratuito en PDF

- Microsoft. (s.f.). *Microsoft MPI (MS-MPI).* Página informativa

- Computación de Alto Rendimiento para Aplicaciones Numéricas (2026), material de cátedra de la Facultad de Ciencias Exactas y Tecnología, Universidad Nacional de Tucumán.

- Estadísticas para todos. Monty Hall. Leer artículo